Collaborative Intelligence: A Survey of Edge-Cloud Split Inference

Yi Gu[1]

Nanjing University of Information Science and Technology, Nanjing, China
202283910033@nuist.edu.cn

Abstract. The rapid proliferation of Deep Neural Networks (DNNs) and Large Language Models (LLMs) has revolutionized artificial intelligence but also introduced significant computational challenges. Traditionally, inference tasks are either offloaded entirely to the cloud, incurring high latency and privacy risks, or executed locally on edge devices, which often lack the necessary hardware resources. Split inference, or collaborative intelligence, emerges as a promising paradigm that partitions neural networks between edge devices and the cloud. This paper surveys the evolution of split inference, from early layer-wise partitioning in CNNs to modern phase-splitting techniques for Generative AI. We analyze the system architecture, discuss key algorithms like Neurosurgeon and SPINN, and evaluate the performance benefits in terms of latency and energy efficiency. Finally, we discuss open challenges such as dynamic network conditions and security implications.

Keywords: Cloud Computing · Edge Computing · Split Inference · LLMs · Collaborative Intelligence.

1 Introduction

Cloud computing has long been the de facto standard for deploying heavy artificial intelligence models due to its virtually unlimited scalability and access to high-end accelerators like NVIDIA A100/H100 GPUs. However, the centralization of AI inference presents critical bottlenecks for modern latency-sensitive applications. Sending high-resolution video streams from autonomous vehicles or smart cameras to a central server introduces round-trip time (RTT) delays that are unacceptable for real-time decision-making. Furthermore, transmitting sensitive user data—such as medical images or personal voice commands—raises significant privacy concerns and bandwidth costs.

Conversely, Edge Computing aims to bring computation closer to the data source. While modern mobile SoCs (System on Chips) have become more powerful, they still lag significantly behind data center GPUs in memory bandwidth and compute capacity. This gap is particularly evident with the rise of Large Language Models (LLMs) like Llama-3 or GPT-4, which require hundreds of gigabytes of VRAM, making purely local execution impossible for high-fidelity models.

Split Inference, also known as Collaborative Intelligence, bridges this gap by distributing the execution of a neural network across the edge-cloud continuum. By partitioning the model—executing the initial layers on the edge device and the deeper, more resource-intensive layers in the cloud—this paradigm leverages the low latency of edge processing and the massive compute power of the cloud. This report explores the advancements in split inference, detailing its architecture, optimization strategies, and its application to the latest generation of AI models.

2 Literature Review

The concept of offloading computation is not new, but the specific application to neural network partitioning has evolved significantly over the last decade.

2.1 Foundational Work: Layer-wise Partitioning

One of the seminal works in this field is *Neurosurgeon* [1]. Kang et al. demonstrated that for many Convolutional Neural Networks (CNNs) like VGG and ResNet, there exists an optimal partition point where the size of the intermediate feature map is smaller than the input data. By identifying this "neck" in the network, Neurosurgeon dynamically partitions the model at runtime to minimize total end-to-end latency. Their work showed that collaborative execution could outperform both cloud-only and mobile-only execution under varying network conditions.

2.2 Progressive and Dynamic Inference

Building on the static partitioning of Neurosurgeon, later works introduced dynamic and progressive strategies. *SPINN* [2] proposed a synergistic progressive inference mechanism. Instead of a single cut, SPINN allows for early exits and progressive transmission of features. It employs a scheduler that accounts for device constraints and network volatility, ensuring that the system can adapt to sudden drops in bandwidth (e.g., a mobile user entering an elevator). This robustness is crucial for real-world deployment of split inference systems.

2.3 Split Inference in the Era of Generative AI

With the advent of Generative AI, the focus has shifted from CNNs to Transformers. The recently published *Splitwise* [3] addresses the unique challenges of Large Language Models. Unlike CNNs, where intermediate data size varies significantly between layers, Transformers have relatively constant hidden state sizes. Splitwise introduces "Phase Splitting," separating the compute-bound prefill phase (which runs better on the cloud) from the memory-bound decoding phase (which can be optimized on edge clusters). This represents the cutting edge of collaborative intelligence, moving beyond simple layer cutting to semantic task distribution.

3 System Architecture

A typical Split Inference system consists of three main components: the Edge Client, the Cloud Server, and the Profiler/Scheduler.

3.1 The Partitioning Engine

The core of the architecture is the Partitioning Engine. It takes the neural network computation graph (DAG) and determines the optimal cut point. This decision is modeled as an optimization problem:

$$\min_{k}(T_{edge}(1...k) + T_{trans}(O_k) + T_{cloud}(k+1...N)) \tag{1}$$

Where T_{edge} is the computation time of layers 1 to k on the device, T_{trans} is the transmission time of the output tensor O_k, and T_{cloud} is the remaining computation time.

3.2 Containerization and Orchestration

In a modern cloud-native environment, the cloud portion of the split model is typically deployed as a microservice using containers (Docker) and orchestrated by Kubernetes. Frameworks like KServe or TorchServe are used to expose the "tail" of the model as a gRPC/REST API. The edge device acts as the client, executing the "head" of the model and sending the intermediate tensor to the KServe endpoint. This decoupling allows the cloud infrastructure to scale independently (using Horizontal Pod Autoscaling) based on the number of incoming inference requests.

3.3 Data Compression and Transmission

To make split inference viable, the intermediate data sent over the network must be compressed. Standard image compression (JPEG) is often unsuitable for feature maps as it introduces loss that degrades model accuracy. Specialized feature compression techniques, such as quantization (reducing precision from FP32 to INT8) and lossless entropy coding, are employed to reduce the bandwidth footprint without compromising the inference quality.

4 Methodology: Implementing Split Inference

To evaluate the effectiveness of split inference, we propose a methodology based on profiling and simulation.

4.1 Model Profiling

The first step is to profile the target model (e.g., ResNet-50 or BERT) on both the target edge hardware (e.g., NVIDIA Jetson Nano or Raspberry Pi) and the cloud instance (e.g., AWS g4dn.xlarge). We measure:

- **Layer-wise Latency:** Time taken to execute each layer.
- **Output Tensor Size:** The data volume generated by each layer.
- **Memory Usage:** Peak RAM consumption.

4.2 Network Simulation

Since real-world networks are volatile, we simulate various network conditions using tools like ‘tc‘ (Traffic Control) on Linux. We test bandwidths ranging from 3G (1 Mbps) to 5G (1 Gbps) and latencies from 10ms to 200ms. This allows us to identify the "break-even point" where split inference becomes superior to pure offloading.

5 Use Cases and Performance Evaluation

5.1 Augmented Reality (AR) and Wearables

In AR applications, headsets must process video feeds to overlay digital information. Offloading everything to the cloud causes motion-to-photon latency, leading to motion sickness. Running everything locally drains the battery in minutes. Split inference allows the headset to perform lightweight feature extraction (e.g., object detection bounding boxes) while the cloud performs heavy rendering or detailed object recognition, balancing battery life with performance.

5.2 Private LLM Assistants

Consider a voice assistant powered by a 70B parameter model. Privacy laws may dictate that the user’s raw voice audio never leaves the device. Using a split architecture, the local device can process the audio and initial embedding layers (anonymizing the features) before sending the abstract representation to the cloud for the heavy reasoning tasks. This "Privacy-Preserving Split Inference" is a major driver for adoption in healthcare and finance.

5.3 Autonomous Vehicles

Self-driving cars generate terabytes of data daily. While critical safety decisions (braking) must be local, auxiliary tasks like "landmark recognition" for map updates can be split. The car processes the raw camera feed to extract key features and uploads only the relevant feature vectors to the cloud, saving massive amounts of cellular bandwidth costs.

6 Cost Analysis

A critical often-overlooked aspect is cost.

- **Bandwidth Cost:** Cloud providers charge for data egress/ingress. By compressing data via split inference, we reduce the billable bandwidth.
- **Compute Cost:** Edge compute is "free" (paid by the user/device owner), while cloud compute is metered. Offloading the first 20% of layers to millions of user devices can significantly reduce the required size of the cloud GPU cluster, lowering the Total Cost of Ownership (TCO) for the service provider.

7 Limitations and Challenges

7.1 Network Volatility

The assumption of a stable connection is the Achilles' heel of split inference. In 5G networks, bandwidth can fluctuate rapidly. If the network speed drops below the threshold required to transmit intermediate tensors, the system performance degrades worse than a purely local execution. Robust fallback mechanisms are required.

7.2 Security of Intermediate Representations

While splitting offers some privacy, it is not a silver bullet. Research has shown that original inputs can sometimes be reconstructed from intermediate feature maps (Inversion Attacks). Securing the communication channel and applying differential privacy noise to the features are active areas of research.

8 Conclusion

The Edge-Cloud Continuum represents the future of scalable AI deployment. By moving away from the binary choice of "Cloud vs. Edge" to a collaborative "Cloud + Edge" model, we can achieve systems that are faster, more private, and energy-efficient. While challenges in network stability and security remain, advancements like Neurosurgeon, SPINN, and Splitwise pave the way for ubiquitous intelligence. As 6G networks and more powerful edge accelerators emerge, split inference will become the standard architecture for the next generation of AI applications.

References

1. Kang, Y., Hauswald, J., Gao, C., Rovinski, A., Mudge, T., Mars, J., Tang, L.: Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. In: Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). pp. 615–629. ACM (2017), https://dl.acm.org/doi/10.1145/3037697.3037698

2. Laskaridis, S., Venieris, S.I., Almeida, M., Leontiadis, I., Lane, N.D.: Spinn: Synergistic progressive inference of neural networks over device and cloud. In: Proceedings of the 26th Annual International Conference on Mobile Computing and Networking (MobiCom). pp. 1–15. ACM (2020), https://dl.acm.org/doi/10.1145/3372224.3419194
3. Patel, P., Choukse, E., Zhang, C., Wiggers, A., Kiyavash, N., Laurenzano, M.A., Ahmadi, B., Reddi, V.J.: Splitwise: Efficient generative llm inference using phase splitting. In: Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA) (2024), https://arxiv.org/abs/2311.18677